

DS	Session	Hashing	Question Information	Level 1	Challenge 98
----	---------	---------	----------------------	---------	--------------

Question description

You are given an array A of length N which is initialised with 0 . You will be given Q queries of two types:

- 1 k : set value 1 at index k in array A
- 2 y : print the smallest index x which is greater than or equal to y and having value 1 . If there is no such index print -1 .

Note: Indexing is 1 based

Constraints

- $1 \leq n \leq 10^9$
- $1 \leq q \leq 5 * 10^5$
- $1 \leq y, k \leq n$

Input Format

First line contains two integers N and Q separated by a space.
The next Q lines contain the type of query (i.e. either a 1 or a 2), then a space, then for type 1 queries integer k and for type 2 queries integer y

Output Format

For each query type 2 , print in new line, the smallest index x which is greater than or equal to y and having value 1 . If there is no such index print -1 .

Explanation for Test case 1

- For first query: $2\ 3$, there is no index greater than or equal index 3 , having value -1 , so the answer is -1 .
- For second query: $1\ 2$, set value -1 at index 2 .
- For third query: $2\ 1$, index 2 is greater than index 1 , having value -1 , so the answer is 2 .
- For fourth query: $2\ 3$, there is no index greater than or equal index 3 , having value -1 , so the answer is -1 .
- For fifth query: $2\ 2$, index 2 is equal to index 2 , having value -1 , so the answer is 2 .

Logical Test Cases

Test Case 1
INPUT (STDIN)
5 5 2 3 1 2 2 1 2 3 2 2
EXPECTED OUTPUT
-1 2 -1 2

Test Case 2
INPUT (STDIN)
5 5 2 1 1 2 2 2 1 2 2 3
EXPECTED OUTPUT
-1 2 -1

Mandatory Test Cases

Test Case 1
KEYWORD
for(i=0;i<q;i++)

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY
7

Test Case 2
TOKEN COUNT
225

Test Case 3
NLOC
38

```
Hashing8.c +
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct Node {
5     int key;
6     int priority;
7     struct Node *left;
8     struct Node *right;
9 } Node;
10
11 Node* create_node(int key) {
12     Node* n = (Node*)malloc(sizeof(Node));
13     n->key = key;
14     n->priority = rand();
15     n->left = NULL;
16     n->right = NULL;
17     return n;
18 }
19
20 void split(Node* t, int key, Node** l, Node** r) {
21     if (!t) {
22         *l = *r = NULL;
23     } else if (key < t->key) {
24         split(t->left, key, l, &t->left);
25         *r = t;
```

```

26     } else {
27         split(t->right, key, &t->right, r);
28         *l = t;
29     }
30}
31
32void merge(Node** t, Node* l, Node* r) {
33     if (!l || !r) {
34         *t = l ? l : r;
35     } else if (l->priority > r->priority) {
36         merge(&l->right, l->right, r);
37         *t = l;
38     } else {
39         merge(&r->left, l, r->left);
40         *t = r;
41     }
42}
43
44int find_val(Node* t, int key) {
45     while (t) {

```

```

46     if (t->key == key) return 1;
47     if (key < t->key) t = t->left;
48     else t = t->right;
49 }
50 return 0;
51}
52

```

main | CH.SC.U4CSE25225 | Hashing8.c

Ln 103, Col 1 Spaces: 4 UTF-8 CRLF C

```

53void insert_key(Node* root, int key) {
54     if (find_val(*root, key)) return;
55     Node *l, *r;
56     split(*root, key, &l, &r);
57     Node* n = create_node(key);
58     merge(&l, l, n);
59     merge(root, l, r);
60}
61
62int lower_bound(Node* root, int key) {
63     int ans = -1;
64     Node* curr = root;
65     while (curr) {

```

```
66     if (curr->key >= key) {
67         ans = curr->key;
68         curr = curr->left;
69     } else {
70         curr = curr->right;
71     }
72 }
73 return ans;
74}
75
76void free_tree(Node* t) {
77     if (!t) return;
78     free_tree(t->left);
79     free_tree(t->right);
80     free(t);
81}
82
83int main() {
84     int n, q;
```

```

85     if (scanf("%d %d", &n, &q) != 2) return 0;
86
87     Node* root = NULL;
88     int i;
89     for(i=0;i<q;i++) {
90         int type, val;
91         scanf("%d %d", &type, &val);
92         if (type == 1) {
93             insert_key(&root, val);
94         } else if (type == 2) {
95             int ans = lower_bound(root, val);
96             printf("%d\n", ans);
97         }
98     }
99
100     free_tree(root);
101     return 0;
102 }
103

```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
5 5
```

```
2 3
```

```
1 2
```

```
2 1
```

```
2 3
```

```
2 2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing8.exe
```

```
-1
```

```
2
```

```
-1
```

```
2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
5 5
```

```
2 1
```

```
1 2
```

```
2 2
```

```
1 2
```

```
2 3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing8.exe
```

```
-1
```

```
2
```

```
-1
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```